

PROSPECTOR - Asteroid Mining Feasibility Engine

COMPLETE PROJECT PLAN

1. PROJECT OVERVIEW

Name: PROSPECTOR **Tagline:** "Which asteroid should humanity mine first?" **Problem:** No unified platform exists that combines orbital mechanics + spectroscopy + economics to evaluate asteroid mining feasibility. **Solution:** A web platform that ingests live NASA data, scores every accessible Near-Earth Asteroid on economic viability, and visualizes mining missions in 3D.

2. CONFIRMED LIVE DATA SOURCES (All Free, No API Keys)

API	Endpoint	Data Provided
SBDB Lookup	<code>ssd-api.jpl.nasa.gov/sbdb.api</code>	Individual asteroid: orbit, physical params, spectral type, diameter, density, albedo
SBDB Query	<code>ssd-api.jpl.nasa.gov/sbdb_query.api</code>	Bulk query 780,000+ asteroids with filters
NHATS	<code>ssd-api.jpl.nasa.gov/nhats.api</code>	119 accessible asteroids with delta-v, trajectory data, launch windows
Close Approach	<code>ssd-api.jpl.nasa.gov/cad.api</code>	Close approach events with distances, velocities, dates

3. TECH STACK

Backend

- **Language:** Python 3.11+
- **Framework:** FastAPI
- **Astrodynamics:** poliastro, astropy
- **ML:** scikit-learn, pandas, numpy
- **Database:** PostgreSQL 15
- **Cache:** Redis
- **Task Queue:** Celery (for background data refresh)

Frontend

- **Framework:** React 18 + Vite
- **3D Engine:** Three.js (via @react-three/fiber)
- **Charts:** Recharts or Chart.js
- **Styling:** TailwindCSS
- **State:** Zustand or React Context
- **HTTP:** Axios

DevOps

- **Containerization:** Docker + Docker Compose
 - **Backend Deploy:** Render / Railway
 - **Frontend Deploy:** Vercel
 - **CI/CD:** GitHub Actions
 - **Docs:** Swagger/OpenAPI (auto from FastAPI)
-

4. FOLDER STRUCTURE

```
prospector/
|-- backend/
|   |-- app/
|   |   |-- __init__.py
|   |   |-- main.py                # FastAPI app entry
|   |   |-- config.py              # Settings, env vars
|   |   |-- database.py            # SQLAlchemy engine + session
|   |   |-- models/                # SQLAlchemy ORM models
|   |   |   |-- __init__.py
|   |   |   |-- asteroid.py        # Asteroid table
|   |   |   |-- orbital_elements.py # Keplerian elements
|   |   |   |-- physical_params.py # Diameter, albedo, spectral type
|   |   |   |-- composition.py     # Estimated mineral breakdown
|   |   |   |-- trajectory.py      # NHATS trajectory data
|   |   |   |-- evs_score.py       # Economic Viability Score
|   |   |   |-- commodity_price.py # Metal/mineral prices
|   |   |-- schemas/               # Pydantic request/response schemas
|   |   |   |-- __init__.py
|   |   |   |-- asteroid_schema.py
|   |   |   |-- trajectory_schema.py
|   |   |   |-- evs_schema.py
|   |   |   |-- composition_schema.py
|   |   |-- routers/               # API route handlers
|   |   |   |-- __init__.py
|   |   |   |-- asteroids.py       # /api/asteroids/*
|   |   |   |-- trajectories.py    # /api/trajectories/*
|   |   |   |-- evs.py             # /api/evs/*
|   |   |   |-- compositions.py    # /api/compositions/*
```

```

|-- market.py          # /api/market/*
|-- mission.py         # /api/mission/*

|-- services/          # Business logic layer
|--   __init__.py
|--   jpl_client.py     # NASA API wrapper
|--   nhats_client.py   # NHATS API wrapper
|--   cad_client.py     # Close Approach API wrapper
|--   delta_v_calc.py   # Delta-V computation
|--   composition_estimator.py # Spectral -> minerals ML
|--   evs_engine.py     # EVS scoring algorithm
|--   market_service.py # Commodity price fetcher
|--   mission_planner.py # Mission timeline generator
|--   data_sync.py      # Background sync with NASA

|-- utils/
|--   orbital_math.py   # Keplerian math helpers
|--   unit_converter.py # AU/km/LD conversions
|--   constants.py      # Physical/astro constants

-- tests/
|-- test_jpl_client.py
|-- test_evs_engine.py
|-- test_composition.py
|-- test_delta_v.py

-- requirements.txt
-- Dockerfile
-- alembic/             # DB migrations
-- alembic.ini

-- frontend/
--   public/
--     textures/        # Planet/asteroid textures
--       earth.jpg
--       sun.jpg
--       mars.jpg
--       asteroid_rough.jpg
--     favicon.ico

--   src/
--     main.jsx
--     App.jsx
--     index.css

--     components/
--       layout/
--         Navbar.jsx
--         Sidebar.jsx
--         Footer.jsx

--       dashboard/
--         DashboardPage.jsx
--         StatsCards.jsx
--         TopTargetsTable.jsx
--         EVSDistributionChart.jsx

--       asteroid/
--         AsteroidDetailPage.jsx
--         PhysicalParamsCard.jsx
--         CompositionBreakdown.jsx
--         TrajectoryPanel.jsx
--         MissionTimeline.jsx
--         EVSScoreGauge.jsx

```

```

|-- visualization/
|   |-- SolarSystem3D.jsx
|   |-- OrbitRenderer.jsx
|   |-- AsteroidMarker.jsx
|   |-- TrajectoryArc.jsx
|   |-- CameraControls.jsx
|   |-- SunLight.jsx
|
|-- market/
|   |-- MarketPage.jsx
|   |-- CommodityPriceChart.jsx
|   |-- MarketImpactSim.jsx
|
|-- ui/
|   |-- GlassCard.jsx
|   |-- ScoreGauge.jsx
|   |-- LoadingSpinner.jsx
|   |-- Tooltip.jsx
|
|-- hooks/
|   |-- useAsteroids.js
|   |-- useTrajectory.js
|   |-- useEVS.js
|   |-- useMarket.js
|
|-- services/
|   |-- api.js                # Axios instance + interceptors
|   |-- asteroidService.js
|   |-- trajectoryService.js
|   |-- evsService.js
|   |-- marketService.js
|
|-- utils/
|   |-- orbitMath.js          # Kepler -> 3D coords
|   |-- formatters.js         # Number/date formatting
|   |-- constants.js
|
|-- store/
|   |-- useAppStore.js        # Zustand global state
|
|-- package.json
|-- vite.config.js
|-- tailwind.config.js
|-- Dockerfile
|-- docker-compose.yml
|-- README.md
|-- .env.example
|-- .gitignore

```

5. DATABASE SCHEMA

Table: asteroids

Column	Type	Description
id	SERIAL PK	Internal ID
designation	VARCHAR(50) UNIQUE	e.g. "433", "2000 SG344"

Column	Type	Description
full_name	VARCHAR(200)	e.g. "433 Eros (A898 PA)"
spkid	VARCHAR(20)	JPL SPK-ID
is_neo	BOOLEAN	Near-Earth Object flag
is_pha	BOOLEAN	Potentially Hazardous flag
orbit_class	VARCHAR(10)	AMO, APO, ATE, IEO
last_synced	TIMESTAMP	Last NASA data refresh

Table: orbital_elements

Column	Type	Description
id	SERIAL PK	
asteroid_id	FK -> asteroids	
epoch	FLOAT	Julian date epoch
eccentricity (e)	FLOAT	
semi_major_axis (a)	FLOAT	AU
perihelion (q)	FLOAT	AU
inclination (i)	FLOAT	degrees
long_asc_node (om)	FLOAT	degrees
arg_perihelion (w)	FLOAT	degrees
mean_anomaly (ma)	FLOAT	degrees
period	FLOAT	days
moid	FLOAT	Min orbit intersection distance (AU)

Table: physical_params

Column	Type	Description
id	SERIAL PK	
asteroid_id	FK -> asteroids	
abs_magnitude (H)	FLOAT	
diameter_km	FLOAT	
diameter_min_m	FLOAT	Estimated min size (m)
diameter_max_m	FLOAT	Estimated max size (m)
albedo	FLOAT	
density	FLOAT	g/cm ³
rotation_period_h	FLOAT	hours

Column	Type	Description
spectral_type_tholen	VARCHAR(10)	S, C, M, X, etc
spectral_type_smass	VARCHAR(10)	

Table: compositions (estimated)

Column	Type	Description
id	SERIAL PK	
asteroid_id	FK -> asteroids	
water_pct	FLOAT	Estimated water %
iron_pct	FLOAT	
nickel_pct	FLOAT	
cobalt_pct	FLOAT	
platinum_pct	FLOAT	
gold_pct	FLOAT	
rare_earth_pct	FLOAT	
silicate_pct	FLOAT	
carbon_pct	FLOAT	
confidence	FLOAT	0-1 confidence in estimate
method	VARCHAR(50)	"spectral_mapping" / "ml_model"

Table: nhats_trajectories

Column	Type	Description
id	SERIAL PK	
asteroid_id	FK -> asteroids	
min_delta_v	FLOAT	km/s
min_duration_days	FLOAT	
launch_date	DATE	
outbound_days	INT	
stay_days	INT	
return_days	INT	
c3	FLOAT	km ² /s ²
departure_v_inf	FLOAT	km/s
n_viable_trajectories	INT	
occ_code	INT	Orbit condition code

Table: evs_scores

Column	Type	Description
id	SERIAL PK	
asteroid_id	FK -> asteroids	
score	FLOAT	0-100
accessibility_sub	FLOAT	Delta-v component
value_sub	FLOAT	Resource value component
feasibility_sub	FLOAT	Mission feasibility component
estimated_value_usd	FLOAT	Total estimated value in USD
mission_cost_usd	FLOAT	Estimated mission cost
roi_ratio	FLOAT	value/cost ratio
computed_at	TIMESTAMP	

Table: commodity_prices

Column	Type	Description
id	SERIAL PK	
metal	VARCHAR(50)	platinum, gold, iron, etc
price_per_kg_usd	FLOAT	
source	VARCHAR(100)	
fetches_at	TIMESTAMP	

6. API ENDPOINTS

Asteroids

```
GET /api/asteroids - List all tracked asteroids (paginated)
  ?sort=evs_score&order=desc
  ?orbit_class=APO
  ?min_diameter=0.1
  ?spectral_type=S
  ?page=1&limit=25

GET /api/asteroids/{designation} - Full detail for one asteroid
GET /api/asteroids/{designation}/orbit - Orbital elements only
GET /api/asteroids/{designation}/physical - Physical params only
GET /api/asteroids/search?q=Eros - Search by name/designation
POST /api/asteroids/sync - Trigger NASA data refresh (admin)
```

Compositions

GET	/api/compositions/{designation}	- Estimated mineral breakdown
GET	/api/compositions/{designation}/value resources	- Estimated USD value of resources
POST	/api/compositions/predict asteroid	- Run ML prediction for an asteroid

Trajectories

GET	/api/trajectories/{designation}	- NHATS trajectory data
GET	/api/trajectories/{designation}/windows	- Available launch windows
GET	/api/trajectories/accessible ?max_dv=6&max_duration=360	- All NHATS-accessible asteroids

EVS (Economic Viability Score)

GET	/api/evs/leaderboard ?limit=20	- Top ranked asteroids by EVS
GET	/api/evs/{designation}	- EVS breakdown for one asteroid
POST	/api/evs/recalculate	- Recalculate all EVS scores
GET	/api/evs/stats	- Distribution stats

Market

GET	/api/market/prices	- Current commodity prices
GET	/api/market/impact/{designation} market	- How mining this asteroid would impact market

Mission Planner

POST	/api/mission/plan body: { designation, launch_year, spacecraft_mass_kg }	- Generate full mission plan
GET	/api/mission/{id}	- Retrieve saved mission plan

7. EVS SCORING ALGORITHM

The Economic Viability Score (0-100) is computed as:

$$\text{EVS} = (W1 * \text{Accessibility}) + (W2 * \text{ResourceValue}) + (W3 * \text{Feasibility})$$

Where:

W1 = 0.30 (accessibility weight)
W2 = 0.45 (resource value weight - most important)
W3 = 0.25 (mission feasibility weight)

Accessibility (0-100):

$$= 100 * (1 - (\text{delta_v} - \text{DV_MIN}) / (\text{DV_MAX} - \text{DV_MIN}))$$

DV_MIN = 3.5 km/s (best known)
DV_MAX = 12 km/s (practical limit)

ResourceValue (0-100):

$$\text{estimated_value} = \text{SUM}(\text{mineral_pct} * \text{mass_kg} * \text{price_per_kg}) \text{ for each mineral}$$

$$= 100 * \log_{10}(\text{estimated_value}) / \log_{10}(\text{MAX_KNOWN_VALUE})$$

Feasibility (0-100):

Based on: mission duration, orbit uncertainty (OCC),


```
spin rate (can we land?), size (big enough to mine?)  
= 25 * duration_score + 25 * occ_score + 25 * spin_score + 25 * size_score
```

8. SPECTRAL TYPE -> MINERAL COMPOSITION MAPPING

Spectral Type	Water	Iron	Nickel	Platinum	Gold	Silicate	Carbon
S (Siliceous)	0.1%	25%	2%	0.005%	0.0001%	60%	2%
C (Carbonaceous)	15%	10%	1%	0.002%	0.0001%	30%	20%
M (Metallic)	0.05%	85%	7%	0.01%	0.001%	3%	0.5%
X (Various)	1%	40%	4%	0.008%	0.0005%	30%	5%
V (Vestoid)	0.1%	20%	1.5%	0.003%	0.0001%	65%	1%
B (B-type)	12%	8%	0.8%	0.001%	0.0001%	35%	22%
D (D-type)	8%	5%	0.5%	0.001%	0.00005%	25%	30%

9. KEY FORMULAS

Mass estimation from diameter and density

If density unknown, use spectral default:

S-type: 2.7 g/cm³

C-type: 1.3 g/cm³

M-type: 5.3 g/cm³

Volume = $(4/3) * \pi * (\text{diameter}/2)^3$

Mass = Volume * density

Mass estimation from absolute magnitude (H) when diameter unknown

diameter_km = $(1329 / \sqrt{\text{albedo}}) * 10^{(-H/5)}$

Default albedo by type:

S-type: 0.20

C-type: 0.06

M-type: 0.10

Estimated resource value

For each mineral m:

yield_kg = mass_kg * mineral_percentage[m]

value_usd = yield_kg * price_per_kg[m]

Total value = SUM(value_usd for all minerals)

Mission cost estimate (simplified)

```
launch_cost = delta_v_factor * spacecraft_mass * cost_per_kg_to_LEO
operations_cost = mission_duration_days * daily_ops_rate
total_cost = launch_cost + operations_cost + fixed_overhead
```

cost_per_kg_to_LEO = \$2,720 (SpaceX Falcon 9 rate)

daily_ops_rate = \$50,000

fixed_overhead = \$50,000,000